Misty II + Foundry Local Conversational Robot

A local, embodied AI experience that combines Misty's physical personality with a Windows companion computer running speech, language, and voice models.

PROJECT OVERVIEW
Goals, capabilities, architecture, and technical specifications

Prepared for project and stakeholder discussions
August 18, 2026

1. Project Overview

Misty is an offline-first conversational robot built for portable demonstrations, technical storytelling, and human-robot interaction experiments. The robot provides the embodied experience; a nearby Windows laptop provides the intelligence.

The system listens for **"Hey Misty"**, records the speaker on the laptop microphone, converts speech to text, generates a short response with a local language model, synthesizes speech, and plays the answer through Misty while coordinating her face, LED, head, and arms.

Core project goals

Local and private by design

Run STT, chat, and TTS locally after setup. No cloud inference or API keys are required for normal operation.

Reliable embodied interaction

Use Misty for physical I/O while moving fragile or compute-heavy intelligence to the companion laptop.

Portable developer demonstrations

Support repeatable setup on local Wi-Fi with one-command start, status, mute, unmute, and safe stop workflows.

Natural conversation

Combine wake-word activation, short spoken answers, optional follow-up turns, expressive faces, light, and movement.

Safe robot operation

Gate movement, monitor hardware state, use full safe reboot only, and release robot resources during shutdown.

Extensible experimentation

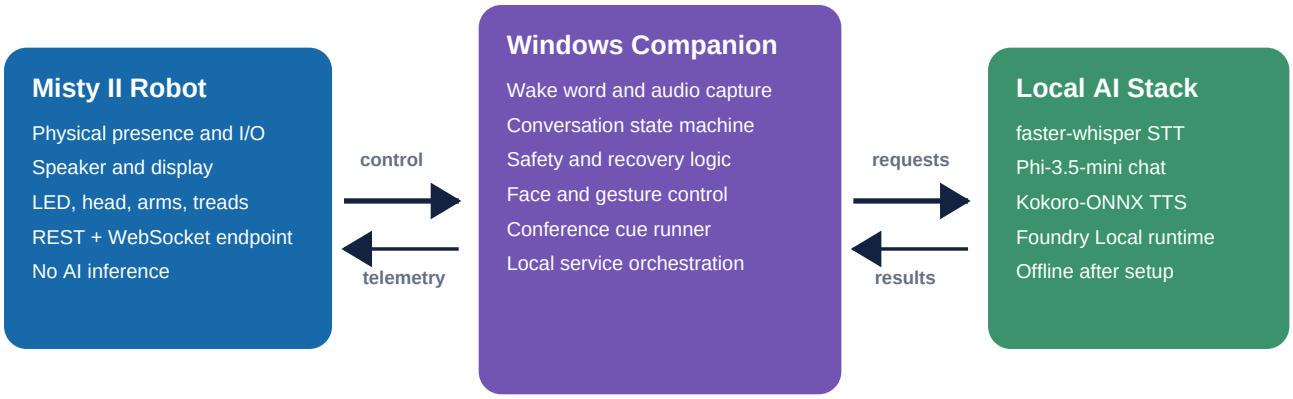
Keep speech, model, face, gesture, conference, and recognition features modular and configuration-driven.

Best-fit scenarios

Developer conferences and booths: live local-AI demonstrations and scripted stage dialog. **Technical workshops:** explain edge AI, model orchestration, and robotics integration. **Prototype research:** test wake words, conversational UX, face tracking, gestures, and safety behavior. **Executive demonstrations:** show private, offline AI in a memorable physical form.

2. System Architecture

The project deliberately separates **embodiment** from **intelligence**. Misty is controlled over the local network using REST and WebSocket APIs; the Windows companion owns audio capture, conversation state, model inference, and recovery logic.



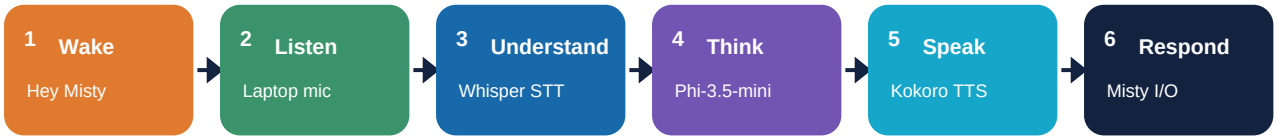
Design decision

Misty II has 2 GB RAM, Windows 10 IoT Core, no general-purpose GPU, final firmware, and a degraded Snapdragon 410 sensory/ML subsystem. On-robot STT, language-model, and TTS inference is therefore not reliable or practical. The companion laptop provides substantially better latency, model flexibility, observability, and recovery while remaining portable.

Area	Specification	Project implementation
Misty II	Physical I/O endpoint	Speaker, display, LED, head, arms, treads, camera access, battery and hazard telemetry.
Misty controller	Python state machine	Coordinates listening, processing, playback, follow-up, movement, expressions, and recovery.
Orchestration service	Flask HTTP service	Provides speech-to-text, prompt orchestration, local chat, text-to-speech, caching, and health endpoints.
Wake-word listener	OpenWakeWord + sounddevice	Continuous non-blocking laptop microphone capture with pause, queue drain, and resume cooldown.
Foundry Local	Local model runtime	Serves Phi-3.5-mini on a dynamically discovered port; no hardcoded service port.
Connectivity	Local Wi-Fi	Misty and the companion must be mutually reachable; normal inference does not require internet access.

3. Conversational Experience

A normal interaction is a coordinated robot experience, not only a chatbot request. Visual and physical cues communicate when Misty is ready, listening, thinking, speaking, or recovering.



Typical interaction

Area	Specification	Project implementation
1. Activation	"Hey Misty"	Laptop OpenWakeWord detects the custom wake phrase; Misty's unsupported built-in keyphrase engine is not used.
2. Listening	Laptop microphone	The controller pauses wake detection, cues the user, captures WAV audio, and can pulse Misty's tally light.
3. Understanding	faster-whisper	Speech is transcribed locally with silence and confidence gates to reduce false follow-up turns.
4. Reasoning	Phi-3.5-mini	Foundry Local generates concise responses within prompt and context budgets.
5. Voice	Kokoro-ONNX	Neural speech is synthesized locally; Windows SAPI through pyttsx3 is the fallback.
6. Embodiment	Misty playback	The answer is played through Misty with coordinated face, LED, and optional safe head/arm expression.
7. Continuation	Follow-up window	When enabled, conversation can continue without repeating the wake phrase; silence or limits re-arm wake detection.

Conversation state model

DISCONNECTED -> IDLE -> RECORDING -> PROCESSING -> PLAYING
PLAYING -> LISTENING -> PROCESSING | REARMING -> IDLE

Additional guarded states cover movement, charging, rebooting, and errors. The controller pauses wake-word inference during Misty's own speech and applies a cooldown before listening again to prevent self-wake.

4. Current Capabilities

Offline conversational AI

Local wake word, speech recognition, Phi-3.5-mini response generation, and neural speech synthesis.

One-command operations

Repository CLI starts services, preflights models and audio dependencies, discovers Misty's IP, reports status, and performs safe shutdown.

Expressive robot feedback

State-linked LED colors, custom face assets, talking animations, and optional bounded head/arm choreography.

Follow-up conversation

Configurable multi-turn listening window with silence handling, context limits, and automatic wake-word re-arming.

Conference Mode

Parses Markdown talk scripts into deterministic audio cues with rehearsal, verification, auto-advance, manual override, and safe stop controls.

Scripted gestures

Conference cues can annotate wave, nod, shrug, glance, face, head, arm, and talking behaviors, including chained gestures.

Companion-side vision

Optional face presence tracking can orient Misty's head; local profile enrollment adds conservative multi-frame identity matching.

Movement safety

Battery, bump, hazard, time-of-flight, sensor freshness, and state checks guard robot movement.

Resilient operation

Non-blocking audio capture, bounded buffering, health checks, watchdogs, TTS fallback, proactive recovery, and explicit error states.

Configurable demonstration modes

Regular conversation, muted/event-safe startup, laptop or Misty conference audio, optional expressions, and feature gates.

Human-facing status language

Area	Specification	Project implementation
Green	Ready / recording cue	Misty is available or inviting the user to speak.
Orange	Wake / preparation	The interaction has started and the system is preparing.
Blue	Processing	Speech and language processing are underway.
Purple	Playback	Misty is speaking.
Cyan	Follow-up listening	The user may continue without another wake phrase.
Yellow / Red	Warning / error	Recovery attention or a fault condition.

5. Technical Specifications

Area	Specification	Project implementation
Robot hardware	Misty II	Qualcomm Snapdragon 820 + Snapdragon 410; 2 GB LPDDR3; 16 GB eMMC; Windows 10 IoT Core.
Robot role	I/O and embodiment	REST + WebSocket controlled speaker, screen, LED, motors, camera, battery, bump, hazard, and distance sensors.
Companion platform	Windows laptop	Runs Python services, Node.js operations CLI, microphone capture, local inference, and robot control.
Wake phrase	Custom "Hey Misty"	OpenWakeWord model on the laptop; default model name hey_misty; threshold 0.85; 2 trigger frames.
Audio input	16 kHz laptop mic	sounddevice capture; configurable device; non-blocking bounded queue; laptop audio is primary.
Speech to text	faster-whisper	Whisper tiny class model, CPU, int8 compute, beam size 1, silence/confidence filtering.
Chat model	Phi-3.5-mini	Foundry model ID Phi-3.5-mini-instruct-generic-cpu:2; served locally through /v1/chat/completions.
Text to speech	Kokoro-ONNX	Primary offline neural voice; default voice af_sky at speed 1.2; pyttsx3/SAPI5 fallback.
Service API	Flask	Default orchestration endpoint http://localhost:5000 with health, orchestration, and TTS surfaces.
Response limits	Bounded prompts	400 user characters; 5,000 context characters; 1,200 context tokens; short answers favored for demos.
Follow-up	Configurable	Enabled by current canonical defaults; 5-second listen cycles; 90-second session window; 12 turn maximum.
Robot maintenance	Proactive recovery	Full Core + Sensory reboot after configured conversation/recording thresholds; never sensory-only reboot.
Conference runtime	Predetermined cues	Markdown script parser, cached WAV manifest, optional presenter VAD auto-advance, manual control, Misty or laptop output.
Privacy	Local processing	Normal conversation audio and model inference remain on the local companion; no cloud API key is required.

Deployment assumption: Misty and the companion laptop share a reachable local network. Internet access may be needed initially to install tools or obtain model assets, but normal prepared operation is offline-capable.

6. Operating Modes

Regular conversation

Wake phrase starts a live STT -> LLM -> TTS interaction. The controller coordinates cues, playback, and optional follow-up turns.

Muted / event-safe

Services and models stay warm while wake-word activation is disabled. Operators can mute or unmute without restarting.

Conference Mode

Misty delivers predetermined lines from a talk script. There is no LLM call at showtime for prepared cues, improving timing and predictability.

Manual conference control

Presenter controls next, replay, previous, pause, resume, auto-advance, and stop. Manual override remains available for stage safety.

Laptop audio conference output

Misty keeps visual and movement choreography while cue audio plays through the laptop or room audio system.

Feature-gated experiments

Face tracking, face recognition, animation, talking head motion, and embodied expressions can be enabled independently after validation.

Operational workflow

Area	Specification	Project implementation
Start	npx . start	Preflight local dependencies and assets, start Foundry Local, load the chat model, start services, and verify Misty connectivity.
Quiet start	npx . start --muted	Keep services ready without accepting wake words.
Control	npx . mute / unmute	Change wake-word availability without restarting the local stack.
Observe	npx . status	Check owned processes and service state.
Conference	npx . conference ...	Dry-run, prepare, verify, and run scripted stage cues.
Stop	npx . stop	Release robot audio and skills, halt movement, return the idle face, turn off the LED, and stop owned services.

What makes the project discussion-worthy

The project is a practical example of **hybrid edge architecture**: constrained hardware remains valuable when paired with a capable local companion. It demonstrates local AI orchestration, real-time audio engineering, graceful degradation, human-readable robot state, safety-aware actuation, and deterministic show control in one system.

7. Safety, Reliability, and Boundaries

Full reboot only

A sensory-only reboot can leave Misty's microphone unusable until physical power cycle. Recovery always reboots Core and Sensory Services together.

Laptop intelligence is required

Misty cannot provide the conversational experience standalone. The companion computer is a required system component.

Built-in keyphrase unsupported

The robot's on-chip keyphrase engine silently degrades. Laptop OpenWakeWord is the supported activation path.

On-chip face ML unavailable

Misty camera capture works, but native face detection and recognition are unreliable. Optional recognition runs on the laptop.

Hardware-aware movement

Movement is constrained by battery state, hazards, bump sensors, distance telemetry, state, and data freshness.

Safe rest state

Shutdown stops legacy keyphrase activity and recording, cancels skills, halts movement, restores the idle face, and turns the LED off.

Local does not mean zero setup

Dependencies and model assets must be installed in advance; Windows ARM64 systems may require x64 Python for live STT.

Demo environment matters

Microphone selection, room noise, network reachability, battery level, and conference audio routing should be rehearsed.

Current boundaries

The system is a demonstration and experimentation platform rather than a production autonomous robot. It is optimized for short, engaging interactions; supervised conference use; and local developer scenarios. Optional motion and recognition features remain gated so operators can validate them against the physical robot and environment.

Discussion summary

Misty turns local AI into a visible, social experience.
The project combines a memorable robot interface with a modern companion-side AI stack, keeps inference private and offline-capable, and treats reliability and physical safety as first-class design requirements.

Primary project references

README.md; docs/ADR-001-companion-device-over-onrobot-inference.md; docs/ADR-002-non-blocking-audio-pattern.md; docs/conference-mode.md; docs/lessons-learned.md; src/windows-orchestration/config_defaults.py